

Wildfire Evacuation Problem

Overview

The problem involves using the Max-Flow Algorithm in a wildfire evacuation scenario with towns, shelters, and roads. Each town has a population needing evacuation, while each shelter has a maximum capacity, and each road has limited carrying capacity. The challenge is the need for simultaneous coordination since sending residents from one town may block another from accessing safety. This is framed as a maximum flow problem, represented as a directed graph where the road network constitutes the edges, and people represent the flow from towns to shelters. The solution consolidates the towns and shelters into a single source and sink, utilizing the Edmonds Karp algorithm, a breadth-first approach to the Ford Fulkerson method.

Time & Space Analysis

Let V be the number of vertices and E be the number of edges in the flow network. The network adds two vertices and one edge per town and per shelter, so:

- $V = T + S + 2 \leq 402$, where the two additional vertices are the super-source and super-sink, and the bound follows from the constraints $T, S \leq 200$; and
- $E = T + S + R \leq 2,400$, comprising one edge per town, one per shelter and one per road, with the bound following from $R \leq 2,000$ (doubled to at most 4,800 stored entries once reverse edges are included).

Time Complexity

The time complexity is $O(V \cdot E^2)$. This is because each iteration of the algorithm performs one Breadth-First Search (BFS) costing $O(V + E) = O(E)$. As BFS selects the shortest augmenting path without decreasing its length between iterations, the number of augmentations is limited to $O(V \cdot E)$. Therefore, this results in a complexity of $O(V \cdot E^2)$. This bound is independent of capacity values, as populations and capacities can reach 10^9 , leading to a maximum possible answer of 2×10^{11} . An implementation relying on the answer's size, rather than the graph's size, is infeasible. On all test cases the accepted solution completes in under 0.1 seconds, while the unit-augmenting variant exceeds the time limit on secret3, consistent with its running time growing with the answer rather than with the size of the graph.

Space complexity

The space complexity is $O(V + E)$. The network is stored as an adjacency map recording, for each edge and its reverse, a capacity and a current flow. The BFS additionally maintains a queue and a parent map, each of size $O(V)$. No structure grows with the magnitude of the capacities.

Solution

The standard maximum flow algorithm assumes exactly one source and one sink, whereas this problem has T origins and S destinations. Following the technique presented in the lectures, two artificial vertices are introduced:

- A **super-source** (vertex 0) with an edge to every town i of capacity p_i . This ensures that no town can send more people than it actually contains.
- A **super-sink** (vertex $T + S + 1$) with an edge from every shelter j of capacity c_j . This ensures that no shelter receives more people than it can house.

Each road from u to v with capacity w becomes a directed edge of capacity w . If several roads join the same pair of locations, their capacities are summed.

Under this construction, every valid evacuation plan corresponds to a feasible flow from the super-source to the super-sink, and vice versa. The value of the maximum flow is therefore precisely the maximum number of people who can be evacuated.

The EdmondsKarp algorithm

The algorithm maintains a residual graph, which records how much additional flow each edge can still accept. It proceeds as follows:

- Search for a path from the super-source to the super-sink using breadth-first search, considering only edges with remaining capacity.
- If no such path exists, terminate: the current flow is maximal.

Otherwise, determine the path's bottleneck, the smallest remaining capacity of any edge along it, and send exactly that amount along the entire path. For every edge on the path, decrease its remaining capacity by the bottleneck and increase the capacity of its reverse edge by the same amount.

Pseudocode:

```
build network: super-source → each town (capacity  $p_i$ ),  
               each shelter → super-sink (capacity  $c_j$ ),  
               each road  $u \rightarrow v$  (capacity  $w$ )  
total ← 0  
while BFS finds a path from super-source to super-sink with spare capacity:  
    b ← smallest spare capacity on that path  
    push b along the path; add b to each reverse edge  
    total ← total + b  
return total
```

The network is stored as nested dictionaries, where `caps[u][v]` and `flows[u][v]` record the capacity and current flow of each edge and its reverse. BFS uses a double ended queue, and a parent map records how each vertex was reached so the augmenting path can be rebuilt from the sink.

Correctness

The reverse edges are essential as they allow a later iteration to cancel or redirect flow committed earlier, so the algorithm is never bound by an early, suboptimal choice. Consider two towns of ten people and two shelters of capacity ten, where Town 1 can reach both shelters but Town 2 can reach only Shelter 1. If Town 1's residents are first routed to Shelter 1, Town 2 appears stranded and a simple greedy approach would answer 10. The residual graph, however, permits the algorithm to push Town 2's residents into Shelter 1 while rerouting Town 1's residents to Shelter 2, achieving the correct answer of 20. The algorithm terminates when no augmenting path remains. By the max-flow min-cut theorem, the absence of an augmenting path implies that the current flow equals the capacity of some cut separating the super-source from the super-sink, and no flow can exceed the capacity of any cut. The flow found is therefore maximal.

Incorrect approaches

The test data is designed to reject three possible but flawed approaches:

- Excluding the super-source and super-sink, and instead computing flow only from the first town to the first shelter. This ignores all other towns and shelters and underestimates the answer.

- Treating roads as bidirectional. The problem specifies one-way roads, and permitting travel in the reverse direction creates routes that do not exist, overestimating the answer.
- Augmenting by one person at a time rather than by the full bottleneck. This approach always produces the correct answer, but requires one iteration per person evacuated. With capacities up to 10^9 , it performs on the order of 10^8 to 10^9 iterations and exceeds the time limit, whereas the bottleneck-based approach requires only a handful of iterations on the same input.

AI Declaration

I utilised Generative AI to help refine the Wildfire Evacuation story to verify if the story aligns with the Max-Flow algorithm as the original story were missing some gaps that I overlooked. Additionally, I used it to verify whether the secret and sample data that I have generated on my own were correct and made sense. Furthermore, I have used the Gen AI to test my understanding of the project's briefing to ensure that I was on the right check by asking clarifications. I have also used it to test my understanding of the Max-Flow Algorithm by sending it my own understanding of the algorithm in my own words as this was one of the recent lecture topics that were taught in that time.